

Contents

guide_01_survey_large_language_model_based_autonomous_agents: A Survey on Large Language Model based Autonomous Agents	2
0. 先定一句话	3
1. 这篇论文的历史位置	3
1.1 从传统 autonomous agent 到 LLM agent	3
1.2 为什么 2023 需要一篇 survey	3
1.3 论文自己的贡献	4
2. 先给新手一张总图	4
3. Construction 第一层: Architecture Design	5
4. Profile Module	5
4.1 Profile 解决什么问题	5
4.2 Profile creation 的三种方法	6
4.3 Profile 的学习重点	7
5. Memory Module	7
5.1 Memory 的作用	7
5.2 Memory structure: unified vs hybrid	8
5.3 Memory format	9
5.4 Memory operations	9
6. Planning Module	11
6.1 Planning 为什么重要	11
6.2 Planning without feedback	11
6.3 Planning with feedback	13
6.4 本仓库对应	14
7. Action Module	14
7.1 Action module 的位置	14
7.2 Action goal	14
7.3 Action production	14
7.4 Action space	15
7.5 Action impact	16
8. Capability Acquisition	16
8.1 Architecture 是 hardware, capability 是 software	16
8.2 With fine-tuning	16
8.3 Without fine-tuning	17
8.4 这对今天使用 AI agent 的意义	18
9. Applications	18
9.1 Social science	19
9.2 Natural science	20
9.3 Engineering	20
10. Evaluation	21
10.1 为什么 agent evaluation 难	21
10.2 Subjective evaluation	21
10.3 Objective evaluation	22
10.4 Protocols	22
10.5 Benchmarks	23
11. Challenges	24
11.1 Role-playing capability	24
11.2 Generalized human alignment	24
11.3 Prompt robustness	24

11.4 Hallucination	25
11.5 Knowledge boundary	25
11.6 Efficiency	25
12. 这篇 survey 没有证明什么	26
13. 对今天 agent framework 选型的意义	26
13.1 不要从框架名开始	26
13.2 框架是模块组合方式	26
14. 本仓库代码对应关系	27
14.1 survey_taxonomy.py	27
14.2 selection_tree.py	27
14.3 langgraph_style.py	28
14.4 llamaindex_style.py	28
14.5 pydantic_ai_style.py	28
14.6 vercel_ai_style.py	28
14.7 claude_agent_sdk_style.py	29
14.8 capstone_same_task.py	29
15. 本地运行路径	29
16. 如何让 AI agent 帮你学这篇	29
16.1 不要让 agent 给你“综述的综述”	29
16.2 让 agent 逼你做映射	30
16.3 让 agent 考 memory reading 公式	30
16.4 让 agent 考 evaluation	30
16.5 让 agent 追问框架选型	30
17. 30 分钟学习任务	30
17.1 第 1 遍: 画四模块图	30
17.2 第 2 遍: 跑 memory reading toy	31
17.3 第 3 遍: 拆一个框架	31
17.4 第 4 遍: 设计一个 evaluation	31
17.5 第 5 遍: 闭卷复述	31
18. 一页纸复习版	32
18.1 四模块	32
18.2 能力获得	32
18.3 评测	32
18.4 挑战	33
19. 闭卷自测	33
20. 你应该带走的直觉	33

guide_01_survey_large_language_model_based_autonomous_agents: A Survey on Large Language Model based Autonomous Agents

原论文: A Survey on Large Language Model based Autonomous Agents

本地原文 PDF: [learning/agent-framework-stack/paper/01_survey_large_language_model_based_autonom](#)

作者: Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhi-Yuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, Ji-Rong Wen

arXiv 初版: 2023。本文 PDF 为 Frontiers of Computer Science 2025 版本。

本导读目标: 把这篇 42 页综述读成一张 agent 架构地图。你读完后应该能用 profile, memory, planning, action 四个模块拆任何 agent framework, 并能把论文 taxonomy 对应到本仓库的 LangChain, LangGraph, LlamaIndex, Pydantic AI, Vercel AI SDK, Claude Agent SDK 等 mock code。

0. 先定一句话

这篇 survey 的核心不是告诉你 “有哪些 agent 项目”。

它真正做的是把 2021 到 2023 年爆发式出现的 LLM-based autonomous agents 归纳成三条主线:

- Construction: agent 怎么搭起来。
- Application: agent 用在哪些领域。
- Evaluation: agent 怎么评测。

其中 construction 又拆成两层:

- Architecture design: 类似 agent 的 hardware。
- Capability acquisition: 类似 agent 的 software。

Architecture design 的核心框架是四个模块:

- Profile。
- Memory。
- Planning。
- Action。

如果你只记住一个图, 就记住这四块。后面所有 LangChain, LangGraph, AutoGen, CrewAI, LlamaIndex, Claude Agent SDK, OpenAI Agents SDK, Pydantic AI 的差别, 本质都可以映射到这四块: 谁负责角色、谁负责记忆、谁负责计划、谁负责行动、谁负责反馈和状态。

1. 这篇论文的历史位置

1.1 从传统 autonomous agent 到 LLM agent

传统 autonomous agent 通常被放在受限环境中训练, 例如游戏、机器人模拟器或强化学习环境。它们的问题是:

- 环境孤立。
- 知识有限。
- 任务范围窄。
- 学习方式和人类差异很大。
- 很难处理开放世界任务。

LLM 出现后, agent 的假设变了。LLM 通过大规模 web data 和大量参数获得广泛世界知识, 并且能用自然语言和人类交互。

这让 agent 设计从 “训练一个策略函数” 转成 “用 LLM 做中央控制器, 再给它 memory, planning, tool use, feedback, role profile 等模块”。

1.2 为什么 2023 需要一篇 survey

2021 到 2023 年间, 很多 agent 工作几乎同时出现:

- WebGPT, 2021-12。

- CoT, 2022-01。
- TALM, 2022-05。
- WebShop 和 Inner Monologue, 2022-07。
- ReAct, 2022-10。
- Toolformer 和 DEPS, 2023-02。
- HuggingGPT 和 AutoGPT, 2023-03。
- Generative Agents 和 AgentGPT, 2023-04。
- Voyager, GITM, ToT, 2023-05。
- RecAgent 和 MIND2WEB, 2023-06。
- ToolBench, CO-LLM, ChatDev, 2023-07。
- AgentSims, 2023-08。

这些工作来自不同方向:

- Tool agents。
- Simulation agents。
- General agents。
- Embodied agents。
- Game agents。
- Web agents。
- Assistant agents。

如果没有统一地图, 学习者很容易只背项目名, 或者只学框架 API, 而不知道每个系统到底在补哪个 agent 能力。

1.3 论文自己的贡献

论文把已有工作系统化整理成:

- 构建方式: 代理架构和能力获得。
- 应用领域: social science, natural science, engineering。
- 评测方式: subjective evaluation 和 objective evaluation。
- 挑战: role-playing, generalized human alignment, prompt robustness, hallucination, knowledge boundary, efficiency。

注意: 这不是一篇提出新模型并跑 benchmark 的论文。它的证据链来自系统归纳、taxonomy、代表性工作对照和失败模式总结。

2. 先给新手一张总图

一个 LLM agent 可以抽象成:

```

user or environment event
-> profile conditions the agent role
-> memory retrieves relevant past state
-> planning decides next steps
-> action module calls tools or produces outputs
-> environment, human, or model feedback returns
-> memory and plan update

```

用更形式化的方式:

```

state_t =
  profile

```

```
+ memory_t
+ task_context_t
+ available_actions

plan_t = planner(state_t, feedback_t)
action_t = actor(plan_t, memory_t, tools)
memory_t_plus_1 = update(memory_t, observation_t, action_t)
```

这不是论文里的神经网络公式，而是读 agent 系统的工程公式。

你读任何 agent framework 都可以问：

- 它如何设定 profile?
- 它如何存 memory?
- 它如何做 planning?
- 它如何执行 action?
- 它如何接收 feedback?
- 它如何获得新能力?
- 它如何评测?

这七个问题就是本篇导读的骨架。

3. Construction 第一层: Architecture Design

论文第 2.1 节提出统一 architecture framework:

```
Profile
  influences Memory
  influences Planning

Memory and Planning
  make the agent situated in a dynamic environment
  let it recall past behavior
  let it plan future actions

Action
  translates decisions into concrete outputs
  interacts with the environment
```

作者明确说，profile, memory, planning 三者共同影响 action。

所以 action 不是孤立的 tool call。一个行动背后往往有：

- 当前 agent 是谁。
- 它记得什么。
- 它计划怎么做。
- 它能调用哪些外部能力。

4. Profile Module

4.1 Profile 解决什么问题

Autonomous agent 通常要扮演具体角色，例如：

- coder。

- teacher。
- domain expert。
- researcher。
- chemist。
- social simulator 中的居民。

Profile module 的目标是指定 agent role，并把这些信息写进 prompt，从而影响 LLM 行为。

论文把 profile 内容分成几类：

- Demographic information: 年龄、性别、职业等。
- Personality information: 性格、心理特征等。
- Social information: 与其他 agent 或人的关系。

新手要注意: profile 不是“系统提示词里的一句 roleplay”。它会影响 memory, planning, action 的整个链条。

4.2 Profile creation 的三种方法

4.2.1 Handcrafting 手工写 profile。例如：

You are a product manager.

You are an introverted person.

You are a senior backend engineer.

优点：

- 灵活。
- 可控。
- 适合少量角色。
- 适合明确分工的 agent team。

缺点：

- 人工成本高。
- 大规模 agent population 不现实。
- 容易带入设计者偏见。

论文举的例子包括 Generative Agents, MetaGPT, ChatDev, Self-collaboration 等。软件开发 agent 里常见的 product manager, architect, engineer, tester 就是 handcrafted profile。

4.2.2 LLM-generation 用 LLM 自动生成 profile。常见流程：

```
define profile generation rules
provide optional seed profiles
ask LLM to generate many profiles
validate or filter generated profiles
```

优点：

- 适合大规模 population。
- 成本低。
- 生成速度快。

缺点：

- 精确控制较弱。

- 可能出现不一致 profile。
- 可能偏离目标分布。

论文举 RecAgent: 先手工创建少量 seed profiles, 再用 ChatGPT 生成更多 agent profiles。

4.2.3 Dataset alignment 从真实数据集里取人类属性, 整理成自然语言 profile。

优点:

- 更贴近真实群体。
- 适合社会模拟。
- 可以反映真实人口分布。

缺点:

- 依赖数据质量。
- 需要隐私和伦理处理。
- 可能继承数据偏差。

论文举例是基于 American National Election Studies 的 demographic background 来给 GPT-3 赋予角色, 并研究它是否能生成类似真实人的结果。

4.3 Profile 的学习重点

你要能回答:

- 这个 agent 的 role 是谁定义的?
- role 是单个 prompt 片段, 还是贯穿 memory/planning/action?
- profile 是否来自真实数据?
- profile 是否可能造成 stereotype 或 bias?
- profile 是否会随着经验更新?

在本仓库里, 对应现代框架时:

- LangGraph 可以把 profile 放进 state。
- Pydantic AI 可以用 schema 验证 role 输出。
- Claude Agent SDK 风格可以把 profile 和 allowed tools 绑定。
- Multi-agent 框架可以让不同 role 互相通信。

5. Memory Module

5.1 Memory 的作用

论文说 memory module 存储 agent 从环境中感知的信息, 并利用记录的 memories 辅助未来行动。

它让 agent 能:

- 积累经验。
- 自我演化。
- 保持一致行为。
- 在复杂任务中更合理、更有效。

这和前一篇 MemGPT 正好接上。MemGPT 是 memory OS 的一个具体系统; 这篇 survey 则从更高层把 memory 分成 structure, format, operation。

5.2 Memory structure: unified vs hybrid

5.2.1 Unified memory Unified memory 只模拟短期记忆，通常通过 in-context learning 实现。Memory 直接写进 prompt。

例子:

- RLP 用内部状态作为对话 agent 的短期记忆。
- SayPlan 用 scene graph 和 environment feedback 指导 embodied task planning。
- CALYPSO 用场景描述、怪物信息和 previous summary 辅助 Dungeons and Dragons。
- DEPS 把 task plans 放进 prompt，引导 Minecraft 行动。

优点:

- 实现简单。
- 最近上下文直接可见。
- 对短任务有效。

缺点:

- context window 限制明显。
- 旧信息会被挤出。
- 不适合长程经验积累。

5.2.2 Hybrid memory Hybrid memory 同时建模短期和长期记忆:

- Short-term memory: 暂存最近感知和当前任务状态。
- Long-term memory: 长期保存重要信息，通常可检索。

例子:

- Generative Agents: short-term context 保存当前情况，long-term memory 保存过去行为和 thoughts。
- AgentSims: prompt 信息是短期记忆，vector database 是长期记忆。
- GITM: short-term 存当前 trajectory，long-term 存成功轨迹总结出的 reference plans。
- Reflexion: short-term sliding window 加 persistent long-term storage。
- SCM: 选择性激活最相关 long-term knowledge。
- SimplyRetrieve: query 是 short-term memory，private knowledge base 是 long-term memory。

优点:

- 适合长程推理。
- 能积累经验。
- 能按需检索。

缺点:

- 检索质量成为瓶颈。
- 写入和删除策略复杂。
- 容易出现重复、过期、冲突 memory。

论文有一个重要 remark: 只用 long-term memory 的 agent 很少见。原因是 agent 总是处在连续动态环境里，连续 action 相关性很高，所以 short-term memory 通常不能省。

5.3 Memory format

论文列了几种 memory storage formats。

5.3.1 Natural language 用自然语言直接保存行为和观察。

优点:

- 灵活。
- 可读。
- 保留语义丰富。

例子:

- Reflexion 用自然语言保存经验反馈。
- Voyager 用自然语言描述 Minecraft skills。

5.3.2 Embeddings 把 memory 编码成向量，便于检索。

优点:

- 检索效率高。
- 能做相似度搜索。

例子:

- MemoryBank 用 embedding vector 检索过去对话片段。

5.3.3 Databases 把 memory 存进数据库，支持精确增删改查。

例子:

- ChatDB 用 SQL 操作 symbolic memory。

优点:

- 可控。
- 可审计。
- 适合结构化事实。

5.3.4 Structured lists 把 memory 组织成 list, tree, triples 等结构。

例子:

- GITM 用 hierarchical tree 结构存 sub-goal action lists。
- RET-LLM 把自然语言转成 triplet phrases 存储。

论文强调: 这些格式不是互斥的。一个真实 agent 常常混用格式。

例如 GITM 的 key 可以是 embedding vector, value 可以是 raw natural language。向量负责检索, 文本负责语义解释。

5.4 Memory operations

论文把 memory operations 分成:

- Memory reading。

- Memory writing.
- Memory reflection.

5.4.1 Memory reading Memory reading 是从 memory 中提取有价值的信息来增强行动。

论文给出一个抽象公式:

```
score(q, m) =
  a * recency_score(q, m)
  + b * relevance_score(q, m)
  + g * importance_score(m)
```

```
m_star = argmax over m in M score(q, m)
```

其中:

- q 是 query, 可以是当前任务或当前上下文。
- M 是全部 memories。
- recency_score 衡量最近程度。
- relevance_score 衡量和 query 的相关性。
- importance_score 衡量 memory 本身重要程度, 不依赖 query。
- a, b, g 是权重。

这条公式是读 memory agent 的关键。不同系统的差别很多时候就是:

- 只看 relevance。
- relevance 加 recency。
- relevance 加 importance。
- importance 由 LLM 打分。
- recency 有时间衰减。

本仓库 survey_taxonomy.py 里有 toy implementation:

```
score = (
  recency_weight * memory.recency
  + relevance_weight * overlap(query, memory.text)
  + importance_weight * memory.importance
)
```

5.4.2 Memory writing Memory writing 是把环境感知到的信息写入 memory。

论文强调两个问题:

1. Memory duplicated.
2. Memory overflow.

Memory duplicated 指新信息和旧 memory 太相似。处理策略包括:

- 合并。
- 计数累积。
- 总结成统一计划。
- 替换旧记录。

Memory overflow 指 memory 满了。处理策略包括:

- 用户显式删除。

- FIFO 覆盖最老记录。
- 摘要压缩。
- 只保留重要记录。

这个问题和 MemGPT 的 queue flush, recursive summary 很接近。

5.4.3 Memory reflection

Memory reflection 让 agent 总结、推断更抽象的高层信息。

论文举 Generative Agents:

- agent 根据最近 memories 生成三个 key questions。
- 用这些 questions 查询 memory。
- 基于检索结果生成五条 insights。
- insights 可以继续分层反思。

例子是从多个低层事件推断出 “某人 dedicated to research” 这类高层洞察。

这和 Reflexion, process reward, self-critique 都有关系。区别是 memory reflection 关注的是把经验沉淀成 reusable insight。

6. Planning Module

6.1 Planning 为什么重要

复杂任务通常不能一步完成。人会把任务拆成子任务，逐个解决。Planning module 的目标就是给 agent 这种能力。

论文按是否有 feedback 分成:

- Planning without feedback。
- Planning with feedback。

6.2 Planning without feedback

这类方法在行动后没有反馈影响未来行为。

6.2.1 Single-path reasoning

Single-path reasoning 把最终任务拆成一条中间步骤链:

```
task
-> step 1
-> step 2
-> step 3
-> answer or action
```

代表方法:

- CoT: 用 few-shot reasoning steps 启发模型逐步推理。
- Zero-shot-CoT: 用 trigger phrase 让模型 step by step。
- RePrompting: 检查每步 prerequisites, 不满足就提示重生成。
- ReWOO: 先生成 plans, 再独立获取 observations, 最后结合。
- HuggingGPT: 拆成 sub-goals, 再调用 HuggingFace models。
- SWIFTSAGE: 用快速模块和深入规划模块结合。

优点:

- 简单。
- 易实现。
- 适合短任务。

缺点:

- 早期错误会传递。
- 没有反馈就很难修正。
- 复杂环境中初始计划可能不可执行。

6.2.2 Multi-path reasoning Multi-path reasoning 让中间步骤形成 tree 或 graph:

step 1

```
branch A
  branch A1
branch B
  branch B1
branch C
```

代表方法:

- CoT-SC: 生成多条 reasoning paths, 用多数答案。
- ToT: 每个 thought 是 tree node, 用 LLM 评估中间步骤, 可用 BFS 或 DFS。
- RecMind: discarded historical information 也用于启发新 reasoning steps。
- GoT: 把 tree 扩展成 graph。
- AoT: 把 algorithmic examples 放进 prompt。
- RAP: 用 world model 和 MCTS 模拟 plan 的收益。

优点:

- 比单一路径更鲁棒。
- 能探索多个候选。

缺点:

- LLM 调用次数更多。
- search space 更大。
- 需要评估函数。

6.2.3 External planner LLM 做高层语义理解, 外部 planner 做正式规划。

代表方法:

- LLM+P: 把任务转成 PDDL, 再由 planner 求解, 再转回自然语言。
- LLM-DP: 把 observation, world state, target objective 转成 PDDL, 再交给 planner。
- CO-LLM: LLM 负责高层计划, 低层 planner 执行动作。

优点:

- 形式化规划更可靠。
- 对 domain-specific planning 有优势。

缺点:

- 需要领域建模。
- 自然语言到 PDDL 可能错。
- 不够通用。

6.3 Planning with feedback

论文说，复杂长程任务里，没有 feedback 的 planning 不够，因为：

- 一开始生成 flawless plan 很难。
- 复杂 preconditions 很难一次考虑全。
- 环境 transition dynamics 不可预测。
- 初始计划可能不可执行。

人类会根据反馈不断修订计划，所以 agent 也需要 feedback。

6.3.1 Environmental feedback 反馈来自客观环境或模拟环境。

代表方法：

- ReAct: thought, act, observation triplets。
- Voyager: 程序执行进度、execution error、self-verification results。
- Ghost: environment states 和 success/failure information。
- SayPlan: scene graph simulator 验证并修订战略。
- DEPS: 给 agent 任务失败的详细原因，而不是只有成功失败。
- LLM-Planner: 遇到 object mismatch 或 unattainable plan 时 grounded replanning。
- Inner Monologue: task completion signal, passive scene description, active scene description。

这类方法和你刚读的 ReAct 直接相连。

6.3.2 Human feedback 反馈来自人类。

好处：

- 对齐人类偏好。
- 缓解 hallucination。
- 解决环境信号无法表达的主观目标。

例子：

- Inner Monologue 让 agent 主动向人类询问 scene description 反馈，再纳入 prompt。

真实产品中的 human-in-the-loop approval, review, correction 都属于这个方向。

6.3.3 Model feedback 反馈由模型自己或辅助模型生成。

代表方法：

- Self-Refine: output, feedback, refinement 循环。
- SelfCheck: 检查各阶段 reasoning steps。
- InterAct: 用不同 LMs 扮演 checker, sorter 等辅助角色。
- ChatCoT: evaluation module 监控 reasoning steps。
- Reflexion: evaluator 根据 trajectory 生成 verbal feedback。

关键区别：传统 RL 可能只有 scalar reward，model feedback 往往是自然语言反馈，更详细，但也可能自我欺骗。

6.4 本仓库对应

langgraph_style.py 是 planning with feedback 的现代工程化 toy:

- State graph。
- Conditional edges。
- Interrupt。
- Resume。
- Checkpoint。

它对应论文里“反馈后修订计划”的思想。复杂 agent 不应该只是 chain，而应该有状态、分支、暂停、恢复和可审计历史。

7. Action Module

7.1 Action module 的位置

Action module 是最下游模块，负责把 agent decision 转成具体结果，并直接和环境交互。

论文从四个角度分析 action:

- Action goal: 行动想达到什么结果。
- Action production: 行动怎么生成。
- Action space: 能做哪些行动。
- Action impact: 行动会产生什么后果。

7.2 Action goal

论文列出三类代表目标。

7.2.1 Task completion 行动用于完成具体任务，例如:

- 在 Minecraft 里制作 iron pickaxe。
- 在软件开发里补全函数。
- 在 WebShop 中完成购物任务。

这类目标通常有明确 success criteria。

7.2.2 Communication 行动用于和其他 agent 或人类沟通，例如:

- ChatDev 多个 agent 讨论软件开发流程。
- Inner Monologue 主动和人沟通，基于 human feedback 调整策略。

这提醒我们: 对 multi-agent 来说，说话本身就是 action。

7.2.3 Environment exploration 行动用于探索未知环境，在 explore 和 exploit 间平衡。

例子:

- Voyager 在 Minecraft 中探索未知 skills，并通过 trial and error refine skill execution code。

7.3 Action production

论文列出两类常见 action production。

7.3.1 Action via memory recollection 根据当前任务从 memory 中抽取信息，然后用任务加记忆生成 action。

例子:

- Generative Agents 在行动前检索 recent, relevant, important memories。
- GITM 查询是否有类似成功经验，如果有就复用旧 action。
- ChatDev 和 MetaGPT 的 conversation history 会影响后续 utterance。

7.3.2 Action via plan following 先生成 plan，再按 plan 执行 action。

例子:

- DEPS 先生成计划，如果没有失败信号就按计划执行。
- GITM 分解 sub-goals，再逐个执行。

实际系统常常混合两者: 先根据 memory 找经验，再根据 plan 执行当前步骤。

7.4 Action space

Action space 可以粗分成:

- External tools。
- Internal knowledge。

External tools 包括:

- APIs。
- Databases and knowledge bases。
- External models。

论文举了很多工具型 agent:

- HuggingGPT 调用 HuggingFace model ecosystem。
- WebGPT 自动生成 query 并从网页取内容。
- Gorilla 生成精确 API arguments，缓解 API call hallucination。
- Toolformer 用 self-supervised learning 学会何时如何调用 tools。
- API-Bank 用多样 API tools 做训练和评测。
- ToolLLaMA 和 ToolBench 聚焦 tool-use data, training, evaluation。
- RestGPT 连接 RESTful APIs。
- TaskMatrix.AI 连接 API ecosystem。
- ChatDB 用 SQL 查询数据库。
- MRKL 和 OpenAGI 连接 expert systems。
- ViperGPT 生成 Python code 并执行。
- ChemCrow 使用 17 个 expert-designed chemistry tools。
- MM-REACT 连接多模态外部模型。

Internal knowledge 包括 LLM 自身的:

- Planning capability。
- Conversation capability。
- Common sense understanding。

一个强 agent 通常两者都要用: internal knowledge 负责理解、规划、交流; external tools 负责计算、检索、执行和验证。

7.5 Action impact

行动的结果不只是输出一段文字。Action 可能:

- 改变环境状态。
- 改变 agent 内部状态。
- 触发新行动。

这一点对安全非常重要。一个会调用 API、写文件、发邮件、运行代码的 agent, 比普通 chatbot 风险高很多。

8. Capability Acquisition

8.1 Architecture 是 hardware, capability 是 software

论文做了一个好比喻:

- Architecture design 类似定义网络结构。
- Capability acquisition 类似学习参数或获得技能。

只有架构不够。Agent 还需要任务特定能力、技能和经验。

Capability acquisition 分成:

- With fine-tuning。
- Without fine-tuning。

8.2 With fine-tuning

8.2.1 Human annotated datasets 用人工标注数据微调 agent。

例子:

- CoH 把 human feedback 转成自然语言比较信息, 用来对齐。
- RET-LLM 用人工构造的 triplet-natural language pairs 微调, 把自然语言转成结构化 memory。
- WebShop 收集 1.18 million Amazon products, 构建模拟购物网站, 并让 13 workers 收集人类行为数据。
- EduChat 用教育场景人工数据微调。

优点:

- 质量高。
- 目标明确。

缺点:

- 昂贵。
- 慢。
- 不容易覆盖全部场景。

8.2.2 LLM generated datasets 用 LLM 生成训练数据。

例子:

- ToolBench 收集 16,464 real-world APIs, 覆盖 49 categories, 从 RapidAPI Hub 获取, 并用 ChatGPT 生成 single-tool 和 multi-tool instructions, 再微调 LLaMA。
- 社交能力相关工作会让多个 agents 在 sandbox 中互动, 收集反馈和解释, 再用作训练数据。

优点:

- 便宜。
- 可扩展。
- 适合多样任务。

缺点:

- 质量可能不如人工。
- 会继承 teacher model 偏差。
- 可能生成看似多样但实际单调的数据。

8.2.3 Real-world datasets 直接用真实世界数据微调。

例子:

- MIND2WEB 收集超过 2,000 open-ended tasks, 来自 137 real-world websites, 覆盖 31 domains。
- SQL-PaLM 用 Spider 和 BIRD 等 text-to-SQL 数据微调 PaLM-2。

优点:

- 接近真实任务。
- 分布更复杂。

缺点:

- 数据清洗难。
- 隐私和许可问题。
- 标注和交互轨迹可能不完整。

8.3 Without fine-tuning

论文把不微调的方法分成:

- Prompt engineering。
- Mechanism engineering。

8.3.1 Prompt engineering 通过自然语言 prompt 描述期望能力。

例子:

- CoT 用中间推理步骤增强复杂推理。
- CoT-SC 和 ToT 也属于这条线。
- RLP 用 agent 对自身和听众 mental states 的 beliefs 提升对话自我意识。
- Retroformer 生成失败反思, 并把反思放回 prompt。

Prompt engineering 的优点是快, 缺点是 prompt robustness 差。

8.3.2 Mechanism engineering 这是论文很重要的概念。

Mechanism engineering 指通过专门模块、工作规则、反馈机制、记忆机制来提升 agent 能力, 而不是直接改模型参数。

论文列了四类。

第一类: Trial-and-error。

agent acts
critic evaluates
feedback is appended
agent revises action or plan

例子: DEPS, RoCo, PREFER。

第二类: Crowd-sourcing。

多个 agents 分别回答, 如果不一致, 就互相吸收意见, 迭代到共识。

这和 multi-agent debate 很接近。

第三类: Experience accumulation。

agent 成功完成任务后, 把 action sequence 或 skill 存入 memory。未来遇到类似任务时检索复用。

例子:

- GITM 保存成功任务 action。
- Voyager 建 skill library。
- AppAgent 通过探索和人类演示构建手机 app knowledge base。
- MemPrompt 保存用户自然语言反馈, 未来相似问题检索。

第四类: Self-driven evolution。

agent 自主设定目标, 通过探索环境和反馈改进自己。

例子:

- LMA3。
- SALLM-MS。
- CLMTWA。
- NLSOM。

8.4 这对今天使用 AI agent 的意义

如果你想用 AI agent 正确学习, 不要只想 “prompt 怎么写”。

你要问:

- 我有没有让 agent 形成 feedback loop?
- 我有没有让它保存经验?
- 我有没有让它用检索回到旧错误?
- 我有没有让它比较多个候选答案?
- 我有没有让它把能力沉淀成代码或 checklist?

这就是 mechanism engineering 视角。它比单次问答更能确保知识进脑袋。

9. Applications

论文把应用分成:

- Social science。
- Natural science。
- Engineering。

9.1 Social science

9.1.1 Psychology LLM agents 可用于心理学模拟、实验、心理支持等。

论文提到一些研究发现 LLM 能生成和人类实验相近的结果，但也可能出现 hyper-accuracy distortion，也就是模型过于完美，不像真实人类。

心理支持 agent 可能帮助用户应对焦虑、孤独、抑郁，但也可能产生有害内容。

学习重点: social simulation 不是越强越好。太强、太理性、太知道答案的 agent 反而不可信。

9.1.2 Political science and economy LLM agents 可用于:

- ideology detection.
- voting pattern prediction.
- political speech analysis.
- simulated economic behavior.

风险是 profile 和 alignment 会强烈影响结果，不能把模拟结果当真实社会。

9.1.3 Social simulation 代表工作包括 Social Simulacra, Generative Agents, AgentSims, S3, CGMI 等。

它们模拟:

- online community.
- virtual town.
- social network.
- classroom scenario.
- harmful information propagation.

这类应用最需关注:

- agent profile 是否真实。
- memory 是否一致。
- 知识边界是否合理。
- 多 agent 互动是否有 emergent behavior。

9.1.4 Jurisprudence 法律 agent 可以辅助 legal decision-making。

论文举:

- Blind Judgement: 多个 language models 模拟多个法官，用 voting 汇总。
- ChatLaw: 中文法律模型，支持 database 和 keyword search，缓解 hallucination。

法律场景是高风险场景。Agent 在这里应该被当作辅助工具，而不是自动决策者。

9.1.5 Research assistant LLM agents 可以帮助:

- 摘要论文。
- 提取关键词。
- 生成研究脚本。
- 发现新的研究问题。
- 写作辅助。

对学习仓库而言，这正是你使用 AI agent 的主战场。但必须有检索、引用、复述、代码实验和自测，否则容易变成“看起来懂”。

9.2 Natural science

9.2.1 Documentation and data management 自然科学研究需要整理大量文献和数据。Agent 可以调用互联网、数据库和工具做：

- question answering.
- experiment planning.
- information extraction.
- compound validation.

例子：

- ChatMOF 从人类文本中提取 metal-organic framework 信息，并规划工具预测性质和结构。
- ChemCrow 使用 chemistry databases 验证化合物表示和危险物质。

9.2.2 Experiment assistant Agent 可以辅助实验设计、规划和执行。

例子：

- 有系统用 LLM 自动设计、规划、执行科学实验，调用互联网和 Python code。
- ChemCrow 用 17 个 chemistry tools 给出实验过程建议，并提示潜在安全风险。

学习重点：自然科学 agent 必须能区分“语言计划”和“真实世界可执行实验”。安全审查不能省。

9.2.3 Natural science education Agent 可用于：

- 实验设计教学。
- 数学探索和证明辅助。
- 自动解题和讲解。
- 编程教育。
- 个性化教育支持。

例子包括 Math Agents, CodeHelp, EduChat, FreeText 等。

9.3 Engineering

工程领域包括：

- Computer science and software engineering.
- Civil engineering.
- Aerospace engineering.
- Industrial automation.
- Robotics and embodied AI.

论文在 software engineering 上举了很多例子：

- ChatDev 用多角色 agent 完成软件开发生命周期。
- MetaGPT 抽象 product manager, architect, project manager, engineer 等角色来提升代码生成质量。
- Self-collaboration framework 把多个 LLM 假设成不同专家。
- LLIFT 用 LLM 辅助 static analysis 和漏洞发现。

- ChatEDA 用于 electronic design automation。

这和本仓库的 agent-code-eval, agent-foundations, agent-framework-stack 紧密相关。

10. Evaluation

10.1 为什么 agent evaluation 难

评测普通 LLM 已经难，评测 agent 更难，因为 agent 有：

- 多轮状态。
- 外部工具。
- 环境交互。
- 长程计划。
- 失败恢复。
- 多 agent 协作。
- 人类主观体验。

论文把评测分成：

- Subjective evaluation。
- Objective evaluation。

10.2 Subjective evaluation

Subjective evaluation 用人类判断 agent 能力。适合没有标准数据集或很难定义量化指标的场景。

10.2.1 Human annotation 让人类 evaluator 直接打分或排序 agent outputs。

例子：

- Generative Agents 让 annotators 提问，评估五个和 agent 能力相关的方面。
- Social Simulacra 让 annotators 判断模型是否能改进 online community rules。

优点：

- 贴近人类体验。
- 能评估 friendliness, believability, usefulness 等主观属性。

缺点：

- 成本高。
- 效率低。
- population bias。
- 一致性难。

10.2.2 Turing test 让 human evaluators 判断输出来自 agent 还是人类。

如果区分不了，说明在特定任务上达到 human-like performance。

但 Turing test 不代表 agent 真懂任务，也不代表安全可靠。

10.2.3 LLM as evaluator 论文提到越来越多工作用 LLM 做 subjective assessment 中介。

例子:

- ChemCrow 用 GPT 评估实验结果, 考虑任务完成和过程准确。
- ChatEval 用多个 agents structured debate 来评价候选模型输出。

你已经在 MT-Bench/Arena 那篇读过 LLM judge 的偏差。这里要把那套警惕带回来: 位置偏差、verbosity bias、自增强、数学弱点、参考答案依赖等都可能影响 agent evaluation。

10.3 Objective evaluation

Objective evaluation 用可计算、可比较、可追踪的量化指标。

论文拆成三个方面:

- Metrics。
- Protocols。
- Benchmarks。

10.3.1 Metrics 代表指标包括:

Task success metrics:

- success rate。
- reward or score。
- coverage。
- accuracy。
- error rate。
- program executability。
- task validity。

Human similarity metrics:

- coherence。
- fluency。
- dialogue similarity。
- human acceptance rate。

Efficiency metrics:

- development cost。
- training efficiency。
- inference cost。
- action latency。

10.4 Protocols

论文列出四种 objective evaluation protocols。

10.4.1 Real-world simulation 在游戏或交互式模拟器中评估 agent, 例如 ALFWorld, IGLU, Minecraft。

优点:

- 有环境反馈。
- 能看 trajectory。
- 能评估任务完成。

10.4.2 Social evaluation 在模拟社会或交互场景中评估 social intelligence, 例如 cooperation, communication, empathy, theory of mind。

10.4.3 Multi-task evaluation 用多个领域任务评估 open-domain generalization。

例子:

- AgentBench。
- ToolBench。
- Mobile-Env。
- WebArena。

10.4.4 Software testing 让 agent 做生成 test cases、复现 bug、debug code、和开发者或工具互动等任务。

指标可以是:

- test coverage。
- bug detection rate。
- patch correctness。
- execution success。

10.5 Benchmarks

论文提到很多 benchmarks:

- ALFWorld。
- IGLU。
- Minecraft。
- Tachikuma。
- AgentBench。
- SockET。
- AgentSims。
- ToolBench, 包含 16,464 RESTful APIs。
- WebShop, 包含 1.18 million real-world items。
- Mobile-Env。
- WebArena。

读 benchmark 时要问:

- 它评测的是 planning, memory, tool use, social behavior, software engineering, 还是 embodied action?
- 它是 single-step 还是 multi-step?
- 它是否需要真实工具?
- 它是否能复现失败轨迹?
- 它是否只测平均成功率, 还是也测成本和风险?

11. Challenges

11.1 Role-playing capability

Agent 需要扮演程序员、研究者、化学家等特定角色。

问题:

- LLM 对常见角色模拟较好。
- 对 web corpus 中少见角色或新兴角色模拟较差。
- LLM 不一定能模拟人类 cognitive psychology 和 self-awareness。

可能解决:

- 收集真实人类数据微调。
- 设计更好的 prompt 和 architecture。

新手要警惕: roleplay 很容易看起来像, 但行为规律未必真实。

11.2 Generalized human alignment

传统 LLM alignment 通常让模型符合统一的安全和价值标准。

但 agent simulation 有时需要模拟不同人类价值, 包括负面行为, 用来研究和预防社会问题。

这带来难题:

- 如果模型过度安全, 模拟不真实。
- 如果模型放开负面行为, 可能产生安全风险。
- 不同应用需要不同 alignment。

论文称之为 generalized human alignment。

这个点很深: agent 不一定总是“越善良越适合模拟”, 但真实系统又不能失控。

11.3 Prompt robustness

Agent 不是一个 prompt, 而是一套 prompt framework:

- profile prompt。
- memory prompt。
- planning prompt。
- action prompt。
- tool schema。
- feedback prompt。

一个模块 prompt 的变化可能影响其他模块。

不同 LLM 对同一 prompt framework 反应也不同。

所以 prompt robustness 是 agent 工程的核心问题。真实工程要有:

- regression tests。
- trajectory replay。
- prompt versioning。
- typed outputs。
- tool call validation。
- failure cases。

这也是 `pydantic_ai_style.py` 这类 typed agent mock 的意义。

11.4 Hallucination

Hallucination 不只是普通 LLM 问题，也是 agent 问题。

Agent hallucination 可能更危险，因为它可能：

- 生成错误代码。
- 调用错误工具。
- 写入错误 memory。
- 做出错误计划。
- 产生安全风险。

论文提到 human correction feedback 可以作为缓解方式。今天还需要配合：

- retrieval grounding。
- tool result verification。
- sandbox。
- approval。
- test execution。

11.5 Knowledge boundary

在 human simulation 中，LLM 可能知道真实人类不知道的信息。

例子：模拟用户选择电影时，真实用户可能没看过某电影内容，但 LLM 训练中可能已经知道。

如果不限知识边界，simulation 会失真。

这是 agent simulation 的关键问题：

- agent 应该知道什么？
- agent 不应该知道什么？
- 哪些知识来自 profile？
- 哪些知识来自 environment observation？
- 哪些知识是模型参数里的泄漏？

11.6 Efficiency

Agent 每个 action 可能需要多次 LLM calls：

- memory extraction。
- memory retrieval。
- plan generation。
- tool call。
- feedback processing。
- reflection。

LLM 自回归生成本来就慢，agent loop 让成本和延迟进一步放大。

这就是为什么 production agent 需要：

- caching。
- smaller model routing。
- parallel tool calls。

- state pruning.
- budget-aware planning.
- early stopping.

12. 这篇 survey 没有证明什么

这篇论文不是 benchmark paper，所以它没有证明某个 agent architecture 最优。

它没有证明:

- profile, memory, planning, action 四模块是唯一划分。
- 每个列出的系统都在同一标准下被公平比较。
- 表里的 taxonomy 能覆盖 2026 年所有框架。
- LLM agents 已经达到 human-level autonomous intelligence.
- multi-agent debate, reflection, memory 一定提升效果。

它证明的是: 当时已有 agent 工作可以被一个统一框架解释, 并且 field 的主要问题集中在 construction, application, evaluation 和 challenges。

13. 对今天 agent framework 选型的意义

13.1 不要从框架名开始

如果你一上来问:

我该用 LangChain 还是 LangGraph?

你已经跳过了论文最重要的分析层。

更好的问题是:

我的 agent 需要什么 profile?

需要 unified memory 还是 hybrid memory?

需要 planning with feedback 吗?

需要 external tools 还是 internal knowledge 就够?

需要 human approval 吗?

需要 subjective eval 还是 objective benchmark?

问完这些, 框架选择才有意义。

13.2 框架是模块组合方式

把现代框架映射到论文 taxonomy:

- LangChain: 强在 chain, runnable, tool ecosystem, 适合一般 action production.
- LangGraph: 强在 state graph, feedback loop, checkpoint, interrupt, 适合 planning with feedback.
- LlamaIndex: 强在 document indexing, retrieval, query engine, 适合 hybrid memory 和 RAG-heavy agent.
- Pydantic AI: 强在 typed output, schema validation, retry, 适合 prompt robustness 和 structured action.
- Vercel AI SDK: 强在 TypeScript, streaming UI, tool loop, 适合产品前端和交互式 agent.
- Claude Agent SDK style: 强在 built-in tools, permissions, hooks, 适合工具执行边界和 human control.

- Selection tree: 把业务约束转成框架选择。

这些不是论文原文列出的 2025 框架，而是本仓库把论文思想接到现代工程栈的桥。

14. 本仓库代码对应关系

14.1 survey_taxonomy.py

这是本次为 guide 新增的 toy module。

它实现了：

- MemoryRecord: 保存 text, recency, importance。
- memory_read_score: 对应论文 memory reading 公式。
- select_memory: 选择最相关 memories。
- AgentDesign: 表示 profile, memory, planning, action, capability acquisition。
- architecture_summary: 把 agent design 映射成 taxonomy notes。
- framework_hint: 根据 taxonomy 推荐一个框架方向。

最小实验：

```
$env:PYTHONIOENCODING='utf-8'
```

```
python learning\agent-framework-stack\src\survey_taxonomy.py
```

你可以改：

- recency_weight。
- relevance_weight。
- importance_weight。
- memory_structure。
- planning。
- action_uses_tools。

然后看选出来的 memory 和 framework hint 是否变化。

14.2 selection_tree.py

这个文件不是论文 taxonomy 的直接复刻，而是现代框架选型决策树。

它问：

- 是否 multi-agent。
- 是否 RAG-heavy。
- 是否 typed_required。
- 是否 streaming_ui。
- 是否 vendor_lock。
- 使用 Python, TypeScript, CSharp 还是 Java。
- 是否 enterprise。
- 团队大小。
- use case。

这对论文里的“先理解模块需求，再选框架”。

14.3 langgraph_style.py

对应 planning with feedback。

它模拟:

- graph nodes。
- conditional edges。
- interrupt。
- resume。
- checkpoint。

这正是复杂 agent 超过普通 chain 的地方。

14.4 llamaindex_style.py

对应 hybrid memory 和 retrieval-heavy action。

它模拟:

- documents。
- nodes。
- vector index。
- query engine。
- source nodes。

这对应论文 memory format 中的 embeddings 和 external knowledge。

14.5 pydantic_ai_style.py

对应 prompt robustness, structured action, typed validation。

它模拟:

- schema。
- required fields。
- type coercion。
- retry。

这在 agent action 输出里非常关键。没有 schema, 工具调用和 JSON 输出很容易漂。

14.6 vercel_ai_style.py

对应 product-facing tool loop。

它模拟:

- generate text。
- tool spec。
- multi-step tool loop。

这和 action space 里的 external tools 相连。

14.7 claude_agent_sdk_style.py

对应 tools, permission modes, hooks。

它提醒你: action impact 不只是输出, 工具执行要有权限和拦截。

14.8 capstone_same_task.py

同一个 search plus summary 任务, 用三种 framework style 跑。

它的学习目的不是比较谁更强, 而是看:

- 同一 agent 能力可以被不同框架抽象。
- 不同框架的 LoC 和 control surface 不同。
- 你选框架是在选状态模型、工具模型、类型模型和 UI 模型。

15. 本地运行路径

验证环境:

```
$env:PYTHONIOENCODING='utf-8'
```

```
python learning\agent-framework-stack\environment\verify_env.py
```

跑专题测试:

```
$env:PYTHONIOENCODING='utf-8'
```

```
python learning\agent-framework-stack\src\tests\test_frameworks.py
```

跑 taxonomy toy:

```
$env:PYTHONIOENCODING='utf-8'
```

```
python learning\agent-framework-stack\src\survey_taxonomy.py
```

跑 capstone:

```
$env:PYTHONIOENCODING='utf-8'
```

```
python -c "import sys; sys.path.insert(0,'learning/agent-framework-stack/src'); from capstone_same_task import main; main()"
```

建议阅读顺序:

1. survey_taxonomy.py
2. selection_tree.py
3. langgraph_style.py
4. llamaindex_style.py
5. pydantic_ai_style.py
6. capstone_same_task.py

16. 如何让 AI agent 帮你学这篇

16.1 不要让 agent 给你 “综述的综述”

不要问:

总结一下 LLM autonomous agents survey。

这会得到一堆名词。

要问:

请用 `profile`, `memory`, `planning`, `action` 四个模块考我。

一次只问一个模块。

每次我回答后，你要指出我漏掉的是 `structure`, `format`, `operation`, `feedback`, `action space`, `evaluation` }

16.2 让 agent 逼你做映射

好问题:

请给我一个 `agent` 例子。

我需要把它拆成 `profile`, `memory`, `planning`, `action`。

如果我只说了框架名，你要追问每个模块由谁负责。

16.3 让 agent 考 memory reading 公式

好问题:

给我三条 `memories`，每条有 `recency`, `relevance`, `importance`。

让我算哪条会被选中。

然后让我解释调高 `importance_weight` 会发生什么。

这能防止你只记住 “`memory` 很重要”，却不会解释 `retrieval policy`。

16.4 让 agent 考 evaluation

好问题:

给我一个 `web agent`。

让我设计 `subjective eval` 和 `objective eval`。

`objective eval` 必须包含 `metric`, `protocol`, `benchmark` 三层。

你如果只能说 `success rate`，就还没读懂评测部分。

16.5 让 agent 追问框架选型

好问题:

我想做一个能长期记忆、需要人工审批、会调用外部工具、会流式输出到网页的 `agent`。

请不要直接推荐框架。

先按 `survey taxonomy` 列需求，再映射到本仓库 `selection_tree.py`。

这就是正确使用 `AI agent` 加速学习: 不是让它替你总结，而是让它迫使你进行分类、映射、计算、复述和代码验证。

17. 30 分钟学习任务

17.1 第 1 遍: 画四模块图

画:

`Profile`

`Memory`

`Planning`

`Action`

`Feedback`

并写出每条边:

- profile 影响 memory。
- profile 影响 planning。
- memory 辅助 planning。
- planning 生成 action。
- action 改变 environment 或 internal state。
- feedback 回到 planning 和 memory。

17.2 第 2 遍: 跑 memory reading toy

运行:

```
python learning\agent-framework-stack\src\survey_taxonomy.py
```

然后手动改权重:

- relevance only。
- recency only。
- importance only。
- all equal。

观察选中的 memory 如何变化。

17.3 第 3 遍: 拆一个框架

选 LangGraph:

- Profile 在哪里?
- Memory 在哪里?
- Planning 怎么表示?
- Action 怎么执行?
- Feedback 怎么进入 graph?
- Evaluation 怎么做?

然后打开 langgraph_style.py 找对应代码。

17.4 第 4 遍: 设计一个 evaluation

任务:

做一个 coding agent, 能读 issue、改代码、跑测试。

你要写:

- Subjective eval: 人类 reviewer 看哪些维度。
- Objective metric: patch pass rate, tests passed, regression count, time cost。
- Protocol: real repo tasks, replay, sandbox execution。
- Benchmark: HumanEval 不够, SWE-style tasks 更接近。

17.5 第 5 遍: 闭卷复述

用 250 字回答:

为什么这篇 survey 把 agent construction 拆成 architecture design 和 capability acquisition?
四个 architecture modules 分别解决什么问题?
agent evaluation 为什么不能只看 success rate?

18. 一页纸复习版

18.1 四模块

Profile:

- agent 是谁
- handcrafting, LLM-generation, dataset alignment

Memory:

- agent 记得什么
- unified or hybrid
- natural language, embeddings, database, structured lists
- read, write, reflect

Planning:

- agent 怎么想步骤
- without feedback: single-path, multi-path, external planner
- with feedback: environment, human, model

Action:

- agent 做什么
- goal, production, space, impact
- tools, databases, APIs, external models, internal knowledge

18.2 能力获得

with fine-tuning:

- human annotated datasets
- LLM generated datasets
- real-world datasets

without fine-tuning:

- prompt engineering
- mechanism engineering
- trial-and-error
- crowd-sourcing
- experience accumulation
- self-driven evolution

18.3 评测

subjective:

- human annotation
- Turing test
- LLM-as-judge style evaluation

objective:
metrics
protocols
benchmarks

18.4 挑战

role-playing capability
generalized human alignment
prompt robustness
hallucination
knowledge boundary
efficiency

19. 闭卷自测

1. 为什么作者说 architecture design 像 hardware, capability acquisition 像 software?
2. Profile 的三种生成方式是什么, 各自风险是什么?
3. Unified memory 和 hybrid memory 的差别是什么?
4. 为什么只用 long-term memory 的 agent 很少见?
5. Memory reading 公式里的三个分数是什么?
6. Memory writing 的两个核心问题是什么?
7. Memory reflection 和普通 summary 有什么区别?
8. Planning without feedback 有哪些策略?
9. Planning with feedback 的三种反馈来源是什么?
10. Action module 的四个分析角度是什么?
11. Toolformer 在 action space 里属于什么位置?
12. Mechanism engineering 包括哪四类?
13. Agent evaluation 为什么要同时考虑 subjective 和 objective?
14. Objective evaluation 的 metrics, protocols, benchmarks 分别是什么?
15. 这篇 survey 列出的六个挑战是什么?
16. 本仓库哪个文件实现了 memory reading toy formula?
17. LangGraph style 对应 planning 的哪个方向?
18. LlamaIndex style 对应 memory 的哪个方向?
19. Pydantic AI style 对应哪个 challenge?
20. 如果你要选框架, 为什么不能只问 “哪个框架最好” ?

20. 你应该带走的直觉

学习 agent framework, 最危险的路径是从 API 开始。

正确路径是:

先画 agent taxonomy
再判断任务需要哪些模块
再选择框架
再写 toy implementation
再设计 evaluation
再让 AI agent 考你复述和改代码

这篇 survey 的价值正在这里: 它把 agent 从 “神奇自动助手” 拆回可工程化的模块。

掌握它之后，你看任何 agent 项目都应该能问：

- 它的 profile 是什么？
- 它的 memory 怎么存、怎么读、怎么反思？
- 它的 planning 是否有 feedback？
- 它的 action space 有哪些工具？
- 它的 capability 是微调、prompt，还是机制工程获得？
- 它用什么评测证明真的有效？
- 它在哪些 challenge 上可能失败？

如果你能这样问，AI agent 就不再是替你糊弄知识的摘要器，而是你主动学习、验证、复现和迁移的加速器。